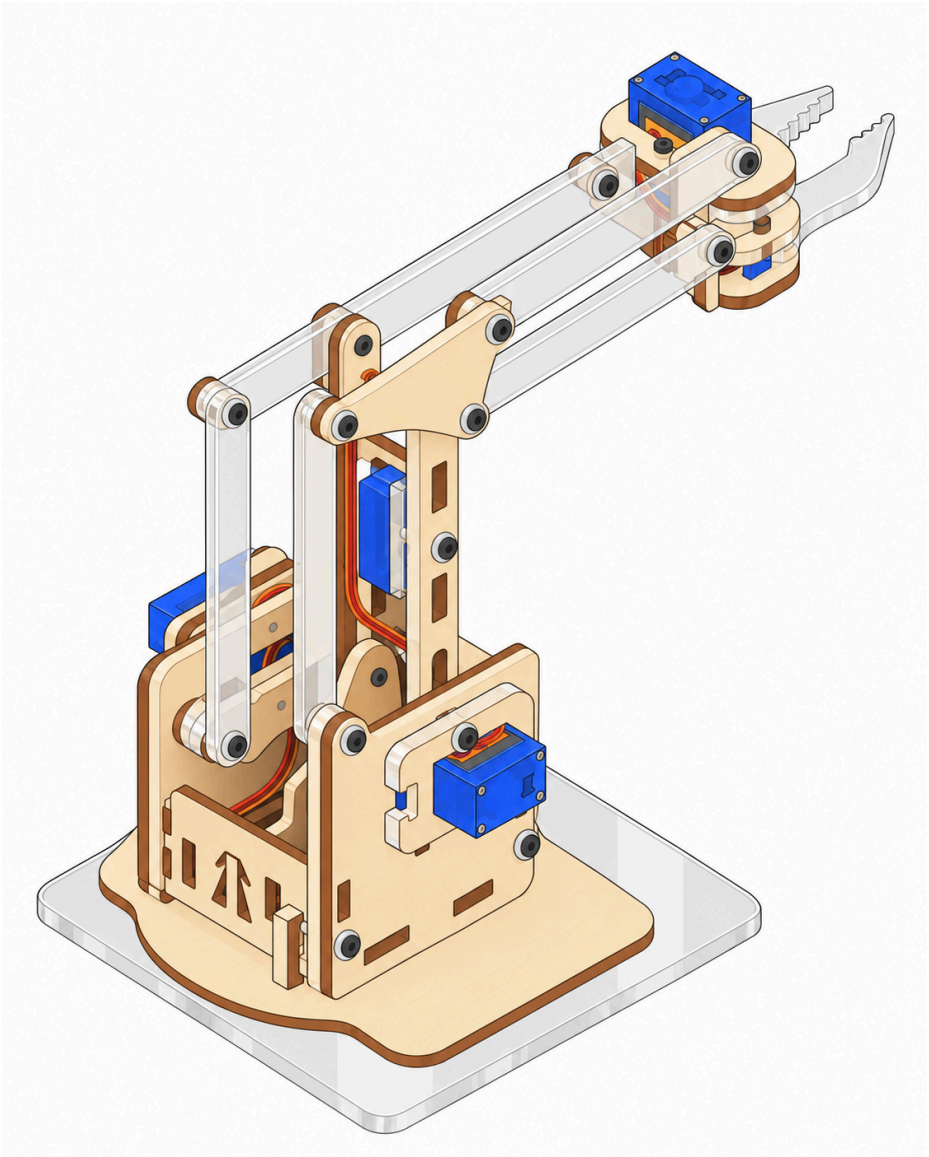
Cyber-Physical Systems
Group 19



Robot Arm Documentation

By: Adam Saint-Jacques, Amber Jiang & Val Aguirre

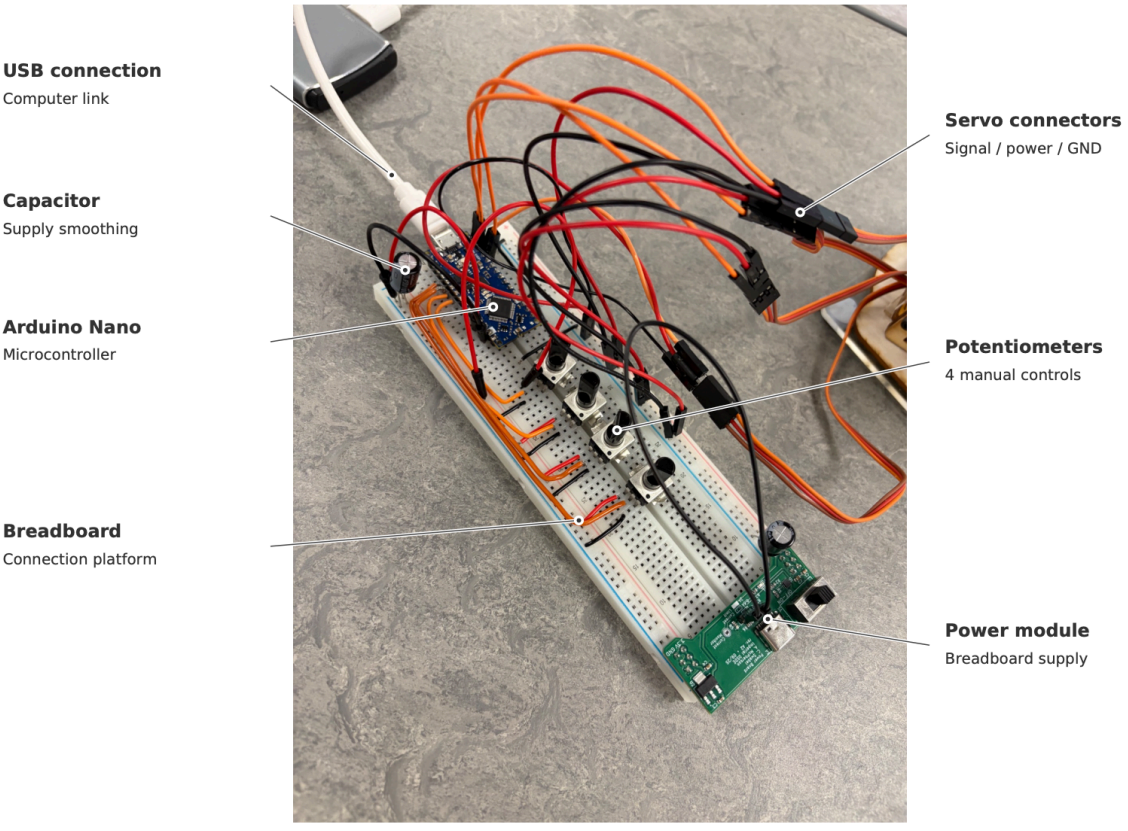
GitHub Repository

1. Overview

For our CPS assessment, we built a 4-servo robotic arm using the resources provided to us. We used the robot arm alongside Arduino, Processing and Wekinator to create a multi-stage cyber-physical system.

Our system uses mouse input to interact with an object on the computer screen. This input is passed to Wekinator, where a trained machine learning model produces one of four possible outputs. The output is then received by Processing and sent to Arduino through serial communication. Arduino interprets the command and controls the corresponding servo motor on our robot arm.

2. Construction



Pin numbers are obscured by wiring in this photograph.

Our system is made up of three main physical components:

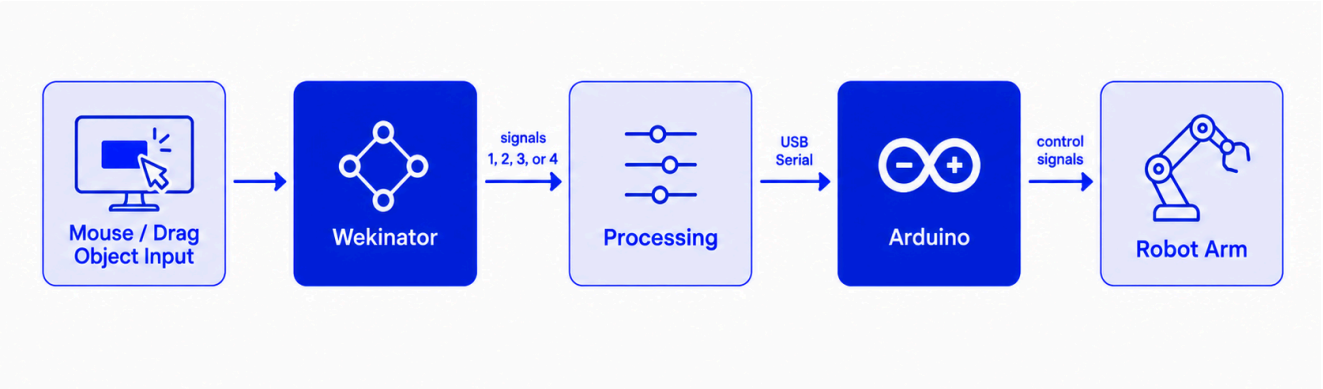
- **Computer:** Runs Processing and Wekinator, allowing us to capture user input, process machine learning outputs, and communicate with Arduino.
- **Robot arm:** Made from wood and acrylic, with four servo motors controlling its movement.

- **Arduino wired with breadboard:** Connects the four servo motors and four potentiometers to the Arduino, allowing us to test and control each motor manually, as well as through our machine learning system.

3. System Architecture

Our physical components work together as part of a larger system architecture.

Each stage has a specific function and can be tested individually, allowing us to identify where a problem might occur without having to test the entire system at once.



System	Function	How to Test
INPUT (Mouse_Dragged_Object)	Captures user input through dragging an object on the screen and sends the input data to Wekinator.	The object should respond to mouse movements, and the input values should change when we interact with it.
Wekinator	Uses the trained machine learning model to interpret the input and produce one of four outputs (1, 2, 3 or 4).	When the model is running, an output of 1, 2, 3 or 4 should be visible in Processing's console “Wekinator class = x;”
Processing	Acts as the bridge between Wekinator and Arduino. It receives the Wekinator output and sends the corresponding command to Arduino through serial communication.	The correct command should be sent to Arduino when Processing receives an output from Wekinator.

Arduino IDE	Receives commands from Processing and controls the appropriate servo motor based on the command received.	Each of the four commands should produce its corresponding servo action.
Robot Arm	Uses four servo motors to produce physical movement based on the commands received from Arduino.	Each servo should respond correctly to its assigned command, and the intended movement should be visible.

4. Unit Tests & Code

For our code, we referenced the source code arduino-osc-blink and processing_osc_to_arduino to expand to work with 4 outputs. We then modified the format to change the outputs from LEDs to the servos in our Robot Arm.

The GitHub contains 3 coding scripts:

1. **Potentiometer_Code** (Unit Test 1)
   Link:https://github.com/valeeriaaguirre-dev/CPS-Group19_RobotArmAssesment/tree/main/1.ArduinoIDE_potentiometercode

   Purpose: For manual control of each servo using potentiometers. Successful run would demonstrate that the servo motors and potentiometers are successfully Connected to the Arduino and communicating with one another.

   ÍBreakdown:

   - Line 1: Pulls the Servo motor library into the program.
   - Line 3: Declares we have 4 “joints” (our custom name for associating one servo per potentiometer - see below for clarification)
   - Line 5: Tells the code the circuit will have 4 servo/potentiometer pairs running (as “joints” = 4)
   - Lines 6 and 7: Declares in an array (list) which pins each servo motor and potentiometer is wired to the Arduino
   - Line 8: Creates a list of storage slots equal to the number of “joints” in the code (4 in this case). This is used to store the values of all 4 servo angles

     Setup ()
   - Lines 11-13: Creates a loop describing “for each joint (0-3), link one servo to the potentiometer on the same index (position within array) one by one. (e.g.,

servo pin #1 links to potentometer pin #1, servo pin #2 links to potentiometer pin #2, … servo pin #4 links to potentiometer pin #4”

- Line 14: Tells Arduino to open serial monitor at 9,600 baud (frequency) to display string messages in the future

loop()

- Lines 18-20: Tells code to measure the value of every potentiometer in the circuit and re-scale its min-max range (0-1023) to the min-max range of the servo motors (0-180 degrees)
- Line 21: Tells Arduino to move each servo motor to the corresponding angle set by its potentiometer
- Line 26: Tells the code to pause for 15 ms after checking the state of each potentiometer-servo joint to prevent overload

Troubleshooting:

1. Verify wiring connections
    a.

2. **Wekinator to Processing (**Unit Test 2)
   Link: https://github.com/valeeriaaguirre-dev/CPS-Group19_RobotArmAssesment/tree/main/3.Processing_to_Wekinator

   Purpose: Translates the classes trained by our ML model into actionable commands Arduino can understand and output to our servo motors.

   Breakdown:
   - Lines 19-20: Imports the data being transmitted by Wekinator into Processing.
   - Line 21: Opens the Serial Monitor, which will write out what class the ML is currently reading in string
   - Line 24:
   - Line 27
   - Line 30: Starts the “class” command at -1 as a reset point

   setup():
   - Line 35: creates a 256 x 150 px canvas
   - Lines 38-39: Prints the ports the host computer contains as a list.
   - Line 43: Tells Processor which port on the host computer the arduino is connected to, and relay messages at 9,600 baud (frequency) [review]
   - Line 46: Listens for new OSC messages from wekinator coming in at 12,000 baud (frequency)

   draw()

- Lines 51-56: Draws a grey gradient on the 256 x 150 canvas (acts as an indicator that **processing is connected to wekinator?**
- 70-78 This code checks whether an incoming OSC message has the address "/wek/outputs" and contains a single floating-point value.
- Line 81-83: Reads the class value from Wekinator (1-4) and prints it in the Serial Monitor
- Line 86: Converts float number into an integer (so description is exact to class)
- Lines 89-92: Ensures integer falls into our possible classes (1-4) and sends that class to Arduino
- Line 94-98: As a failsafe, if the number doesn't fall into our possible classes, prints out "invalid class" to processor serial monitor
- Line 110: Sends a new command to arduno
- Lines 113-122: Ensures Processing only sends a new command to Arduino if the state changes

3. **Processing to Arduino** (Unit Test 3)
   Link: https://github.com/valeeriaaguirre-dev/CPS-Group19_RobotArmAssesment/tree/main/2.ArduinoIDE_to_processing

Purpose: Links all three systems by triggering our Wekinator ML classes to via Prcoessing into actionable outputs to our servo motors

Breakdown:

```
/*
  Arduino - 4 Servo Robot Arm Test

  Receives serial commands (1, 2, 3, 4)
  from Processing and moves the servos.

  For use with Processing / Wekinator.
*/

#include <Servo.h>

// Number of servos
const int Num_servo = 4;

// Create four Servo objects
Servo myservo[Num_servo];

// Pins connected to the servo signal wires
int servoPin[Num_servo] = {12, 11, 10, 9};

// Position of each servo
int pos[Num_servo] = {90, 90, 90, 90};

// Incoming command from Processing
int servoCode = -1;
```

Purpose
Receive commands from Processing and set the target angles of four positional servos.

Signal flow
Wekinator → Processing → Arduino → servos.

Servo.h
The library provides attach() and write().

Num_servo = 4
Four servos are used. const means this value cannot be reassigned.

myservo[]
Four Servo objects. Array indices start at 0: myservo[0] is servo 1 and myservo[3] is servo 4.

servoPin[]
The signal pins are D12, D11, D10 and D9, in that order.

pos[]
The four initial target angles are 90 degrees. These are requested angles, not measured positions.

servoCode = -1
An initial value indicating that no command has been received.

Troubleshooting
Match each signal wire to its listed pin. Use an external supply suitable for the servos and combined load. Connect servo grounds, supply negative and Arduino GND together.

```
void setup() {

  // Start serial communication with Processing
  Serial.begin(9600);

  // Attach each Servo object to its Arduino pin
  for (int i = 0; i < Num_servo; i++) {

    myservo[i].attach(servoPin[i]);

    // Start each servo at 90 degrees
    myservo[i].write(pos[i]);
  }
}


void loop() {

  // Is Processing sending us something?
  if (Serial.available() > 0) {

    // Read one byte from Processing
    servoCode = Serial.read();

    // Choose a robot-arm position
    switch (servoCode) {
```

setup(): runs once
Start serial communication at 9600 baud, attach the servos and send their initial targets.

How the for loop works
Start i at 0. Repeat while i is less than 4. Add 1 after each round. The indices are 0, 1, 2, 3.

Expanded equivalent at startup

```
// Round 1: i = 0
myservo[0].attach(12);
myservo[0].write(90);

// Round 2: i = 1
myservo[1].attach(11);
myservo[1].write(90);

// Round 3: i = 2
myservo[2].attach(10);
myservo[2].write(90);

// Round 4: i = 3
myservo[3].attach(9);
myservo[3].write(90);
```

Startup check
All four servos receive a 90-degree target immediately. Check mechanical clearance before powering on.

loop(): repeats continuously
Serial.available() checks for waiting bytes. Serial.read() reads one byte into servoCode.

Debug
Processing must use the correct port and 9600 baud. Close Serial Monitor before Processing opens the port.

```
    case 1:
      pos[0] = 30;
      pos[1] = 30;
      pos[2] = 30;
      pos[3] = 30;
      break;

    case 2:
      pos[0] = 60;
      pos[1] = 60;
      pos[2] = 60;
      pos[3] = 60;
      break;

    case 3:
      pos[0] = 90;
      pos[1] = 90;
      pos[2] = 90;
      pos[3] = 90;
      break;

    case 4:
      pos[0] = 120;
      pos[1] = 120;
      pos[2] = 120;
      pos[3] = 120;
      break;
```

Each case selects a whole-arm pose
Command 1 does not select only servo 1. It sets the targets for all four servos.

Original mapping
1: all four targets = 30 degrees
2: all four targets = 60 degrees
3: all four targets = 90 degrees
4: all four targets = 120 degrees

Assigning versus moving
pos[0] = 30 stores a number. The later write() loop sends this target to the servo.

break
Exit the switch after the selected case. Without it, execution continues into the following case.

Change a pose
Edit pos[0] through pos[3] inside a case and upload again. The four angles can be different.

Example only

```
pos[0] = 40;  // servo 1
pos[1] = 60;  // servo 2
pos[2] = 90;  // servo 3
pos[3] = 110; // servo 4
```

Choose angles within the safe range of your assembled arm.

Debug: numeric byte versus text
In Processing, port.write(1) sends numeric byte 1. port.write("1") sends character code 49, which does not match case 1. Typing 1 in Serial Monitor also sends text.

```
      default:
        // Do nothing if an unexpected command arrives
        break;
    }

    // Send the new positions to all four servos
    for (int i = 0; i < Num_servo; i++) {
      myservo[i].write(pos[i]);
    }

    delay(10);
  }
}
```

default
Unexpected byte values do not change pos[]. The write loop still sends the previously saved targets.

Send the four targets
This loop runs for i = 0, 1, 2, 3. Each servo receives the target stored at the same index.

Expanded equivalent

```
myservo[0].write(pos[0]);
myservo[1].write(pos[1]);
myservo[2].write(pos[2]);
myservo[3].write(pos[3]);
```

The lines correspond to servos 1, 2, 3 and 4. After case 2, all four calls send 60.

Why no attach() here?
The pin assignments were already made in setup(). This loop only updates targets.

Timing
write() does not wait for arrival. Commands are sent quickly, so movements may overlap. delay(10) pauses for 10 milliseconds; it does not create slow movement.

No new data
The last targets remain active. Servo control signals continue through the Servo library.

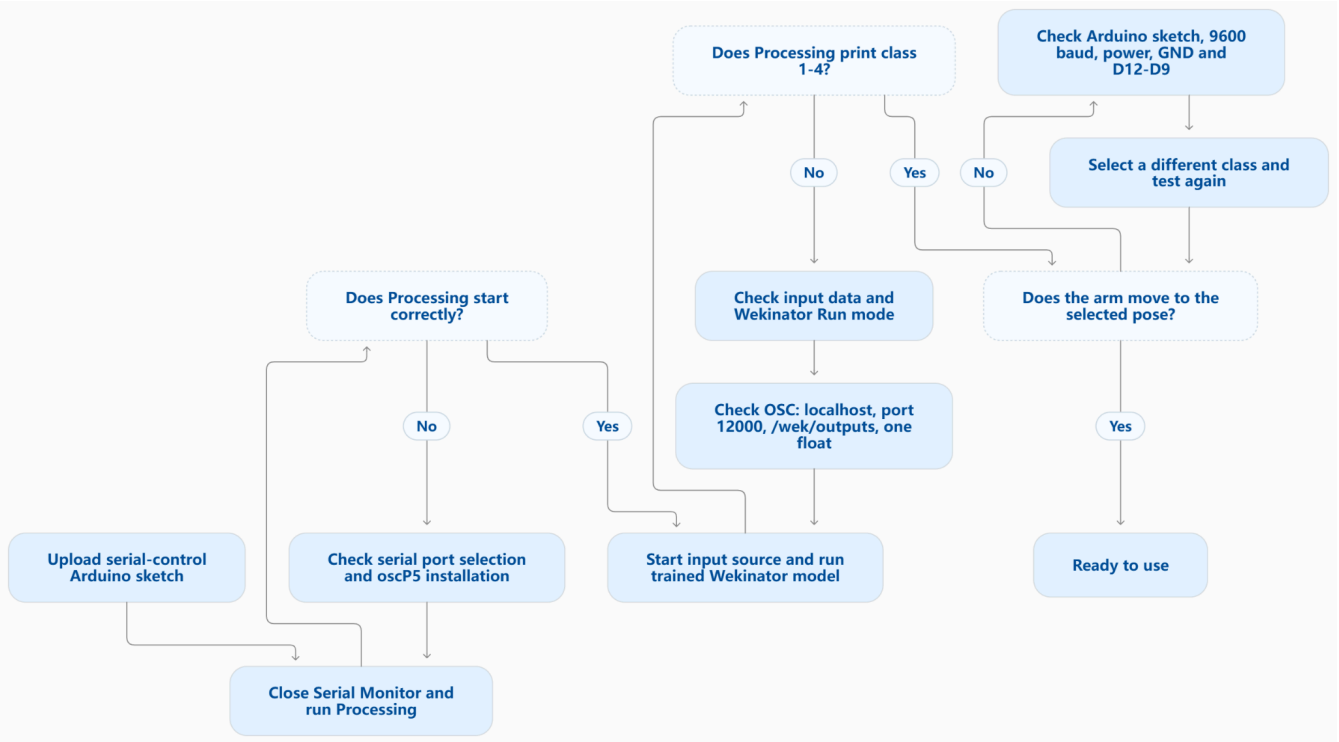
Troubleshooting
No movement: check power, shared GND, signal pins, serial port and numeric-byte sending.

Jitter or resets: check supply capacity, wiring and mechanical load.

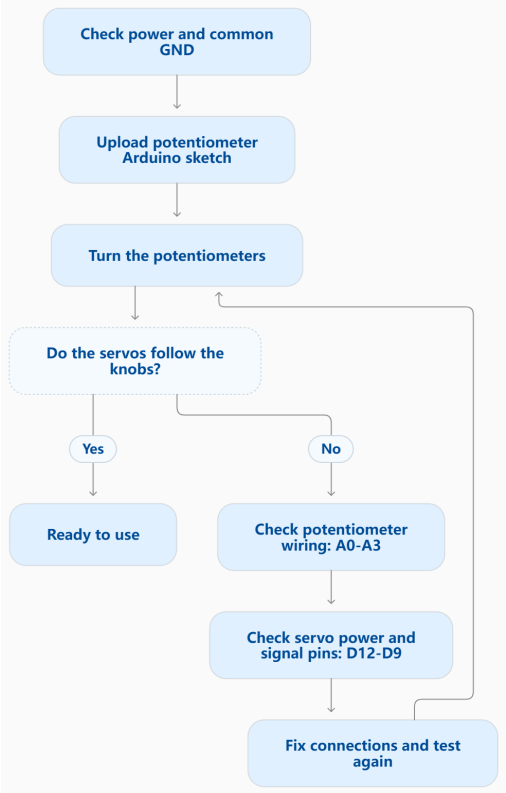
Temporary debugging
After Serial.read(), add Serial.println(servoCode); and read the returned serial data in Processing to inspect received values. Do not open Serial Monitor at the same time.

5. Troubleshooting

Computer Control — Wekinator



Manual Control — Potentiometers



6. Wiring Diagram

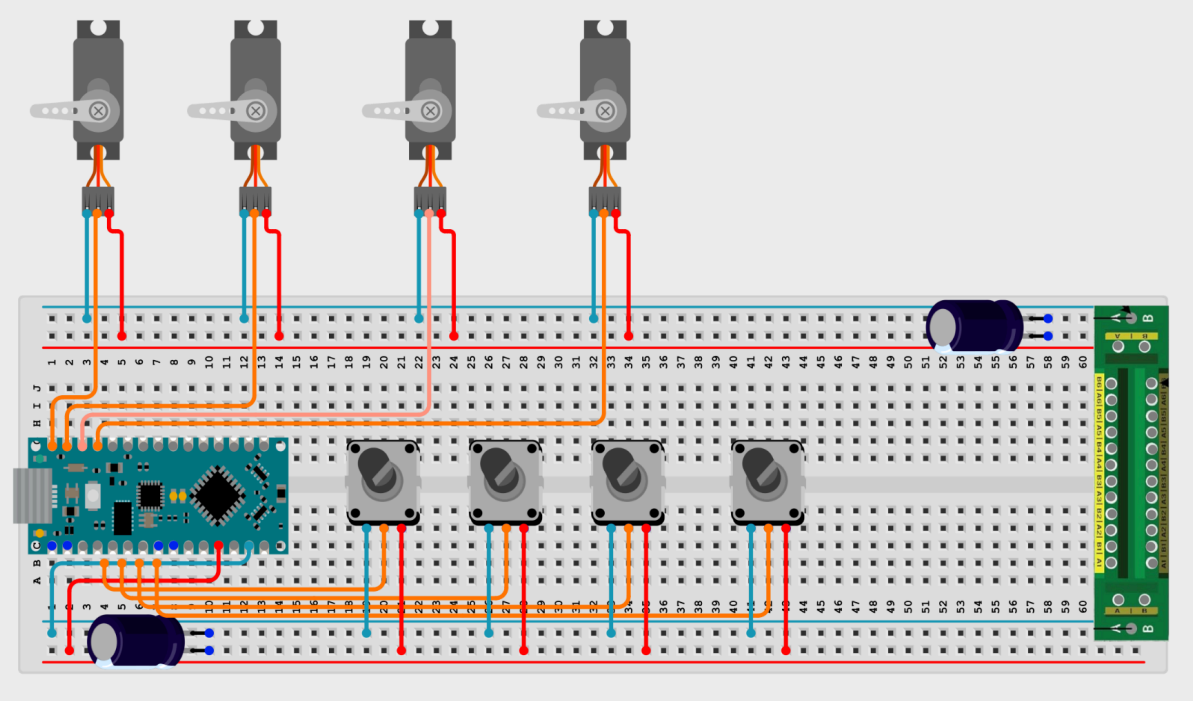


Diagram created on Cirkut Designer